FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Carrera: Lic. en Ciberdefensa

Materia
Sistemas Operativos II

Unidad 3 / Semana 6

Año: 2025

Contenidos

Tipos y niveles de planificación de procesos. Algoritmos de planificación en sistemas por lotes e interactivos. Sistemas apropiativos y no apropiativos.

Índice

1. Planificación de procesos (Scheduling) 3
1.1 Planificador de CPU 5
1.1.1 Puntos de Activación del Planificador 5
1.1.2 Planificación No Apropiativa y apropiativa 5
1.1.3 El despachador (dispatcher) 6
1.2 Niveles de planificación de procesos 7
1.2.1 Planificación a largo plazo (Long-Term Scheduling) 7
1.2.2 Planificación a medio plazo (Medium-Term Scheduling) 8
1.2.3 Planificación a corto plazo (Short-Term Scheduling) / Despachador (Dispatcher) 8
1.2.4 Ejemplo de flujo de planificación 9
1.3 Algoritmos de Planificación de Procesos 10
1.3.1 FCFS (First-Come, First-Served) 11
1.3.2 SJF (Shortest Job First) 11
1.3.2 SRTF (Shortest Remaining Time First) 12
1.3.3 RR (Round Robin) 14
1.3.4 Planificación por prioridades 15
1.4 Planificación de procesos en sistemas multiprocesador 17
1.4.1 Objetivos de la planificación en multiprocesadores 17
1.4.2 Estrategias de planificación 17
1.4.3 Afinidad al procesador 18
1.4.4 Equilibrio de carga 18

1. Planificación de procesos (Scheduling)

Comenzaremos explicando los fundamentos que permitirán entender cómo los sistemas operativos maximizan el rendimiento y gestionan los procesos en sistemas con un solo procesador.

Los sistemas multiprogramados, con frecuencia tienen varios procesos o hilos que compiten por la CPU al mismo tiempo. Esta situación ocurre cada vez que dos o más de estos procesos se encuentran al mismo tiempo en el estado listo. Si sólo hay una CPU disponible, hay que decidir cuál proceso se va a ejecutar a continuación. La parte del sistema operativo que realiza esa decisión se conoce como **planificador de procesos** y el algoritmo que utiliza se conoce como algoritmo de **planificación**.[1]

La planificación de procesos tiene como propósito fundamental **optimizar la utilización del procesador** y **mejorar el rendimiento global del sistema**. Los conceptos que se detallan a continuación permiten comprender la dinámica de ejecución de los procesos y constituyen los fundamentos teóricos esenciales para el diseño, implementación y evaluación de algoritmos de planificación eficientes.

a. Máxima utilización de la CPU con multiprogramación

La **multiprogramación** es la capacidad de un sistema operativo para mantener múltiples procesos en memoria principal al mismo tiempo. El objetivo es maximizar la **utilización de la CPU**, que es el porcentaje de tiempo que la CPU está ocupada realizando trabajo útil. En lugar de dejar que la CPU se quede inactiva cuando un proceso debe esperar por una operación de entrada/salida (E/S), la multiprogramación permite que el planificador de la CPU cambie a otro proceso que esté listo para ejecutarse. De esta manera, se evita que la CPU se encuentre ociosa, logrando así un uso más eficiente de los recursos del sistema y, por tanto, una mayor productividad.

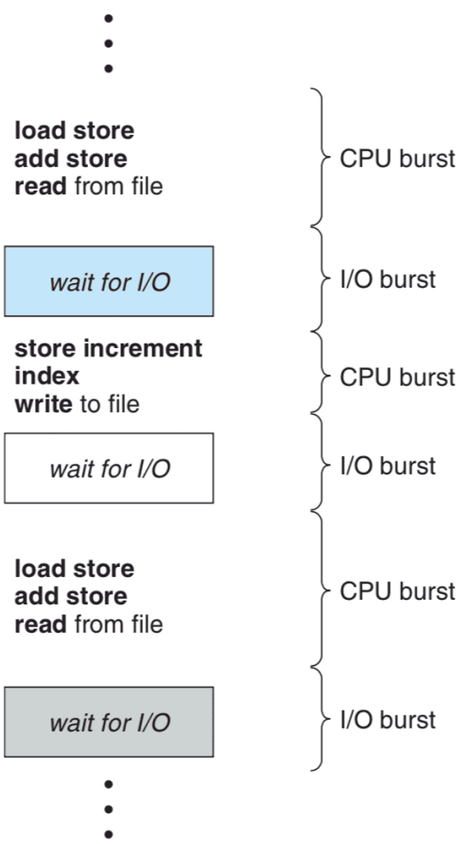
b. Ciclos de ejecución de CPU y esperas en E/S

La ejecución de un proceso se puede ver como una serie de ciclos repetidos, alternando entre la ejecución en la CPU y la espera por operaciones de E/S.

[1] Tanenbaum, Andrew (2009), Sistemas Operativos Modernos. Tercera edición

- **Ráfaga de CPU (CPU Burst):** Es el tiempo que un proceso necesita para ejecutar instrucciones en la CPU sin interrupción.
- **Ráfaga de E/S (I/O Burst):** Es el tiempo que un proceso pasa esperando que se complete una operación de E/S, como la lectura de un archivo o la comunicación con un dispositivo externo.

Esta naturaleza cíclica es la razón por la que la multiprogramación es tan efectiva. Cuando un proceso entra en una fase de E/S, el sistema operativo lo saca de la CPU y cede el control a otro proceso que esté listo, optimizando el uso del procesador.



Secuencia de alternancia de ráfagas de CPU y E/S.

c. Distribución de Ráfagas de CPU

La duración de las ráfagas de CPU no es uniforme. La **distribución de ráfagas de CPU** es un análisis estadístico de la duración de estos ciclos de CPU en un sistema. Esto significa que:

- Hay un gran número de ráfagas de CPU muy cortas.

- Hay un pequeño número de ráfagas de CPU muy largas.

Este conocimiento es crucial para diseñar algoritmos de planificación de la CPU eficientes. Por ejemplo, los planificadores que favorecen las ráfagas cortas (como el algoritmo Shortest Job First) pueden lograr un excelente tiempo de espera promedio, ya que la mayoría de los procesos se procesan rápidamente.

1.1 Planificador de CPU

El planificador es el componente del sistema operativo que se encarga de elegir, de entre todos los procesos que están listos para ejecutarse, a cuál se le asignará la CPU. Su objetivo es asegurar que la CPU esté siempre ocupada con tareas útiles para maximizar su eficiencia.

1.1.1 Puntos de Activación del Planificador

El planificador de la CPU toma el control para asignar el procesador. Estos momentos definen si un sistema es **apropiativo** o **no apropiativo**.

1. **Conmutación de 'ejecutando' a 'espera':** Esto ocurre cuando un proceso, al estar en la CPU, solicita una operación de E/S (como leer un archivo) y debe esperar a que se complete. El proceso libera la CPU voluntariamente. El planificador aprovecha este momento para asignar la CPU a otro proceso.
2. **Conmutación de 'ejecutando' a 'listo':** En este caso, el proceso en ejecución es interrumpido, por ejemplo, porque se acabó su tiempo asignado (un "quantum" de tiempo) o porque llegó un proceso de mayor prioridad. Se le obliga a liberar la CPU y se coloca en la cola de procesos listos.
3. **Conmutación de 'espera' a 'listo':** Esto sucede cuando un proceso que estaba esperando por E/S termina su operación. El proceso pasa a la cola de listos y el planificador debe decidir si debe interrumpir al proceso que actualmente usa la CPU para darle el turno al que acaba de estar listo.
4. **Terminación del proceso:** El proceso en ejecución finaliza su tarea por completo y libera la CPU. El planificador debe seleccionar el siguiente proceso de la cola de listos para ejecutar.

1.1.2 Planificación No Apropiativa y apropiativa

a. Planificación No Apropiativa (Non-preemptive)

En este modelo, una vez que un proceso comienza a ejecutarse, mantiene el control de la CPU de manera ininterrumpida hasta que se cumplen una de dos condiciones: el proceso termina su tarea o libera voluntariamente la CPU para realizar una operación de entrada/salida (E/S). En esencia, un proceso no puede ser forzado a ceder el procesador. Este enfoque es más simple de implementar y reduce el número de **cambios de contexto**, lo que puede ser beneficioso para tareas largas y no interactivas, como las que se encuentran en los sistemas por lotes. Sin embargo, puede llevar a que un proceso muy largo monopolice la CPU, haciendo que otros procesos de alta prioridad o más cortos tengan que esperar de forma indefinida, lo que se conoce como **inanición** (starvation).

b. Planificación Apropiativa (Preemptive)

A diferencia del modelo no apropiativo, en la planificación apropiativa, el planificador de la CPU puede interrumpir un proceso que se está ejecutando para asignarle el control a otro proceso. Esta interrupción puede ocurrir por varias razones, como la llegada de un proceso con una prioridad más alta, o porque el proceso actual ha excedido su tiempo máximo de ejecución (conocido como **quantum**). La capacidad de interrumpir procesos es fundamental para los sistemas interactivos y de tiempo real, ya que garantiza una **capacidad de respuesta** (responsiveness) rápida y justa para todos los usuarios y tareas. Si bien este enfoque introduce una sobrecarga adicional debido a los constantes cambios de contexto, la equidad y la capacidad de respuesta que proporciona lo convierten en el modelo dominante en los sistemas operativos modernos como Linux y Windows.

1.1.3 El despachador (dispatcher)

El módulo **despachador** es el componente de un sistema operativo que implementa las decisiones tomadas por el **planificador a corto plazo**. Su tarea principal es transferir el control de la CPU al proceso que ha sido seleccionado para ejecutarse. Este proceso implica varias acciones críticas:

- **Cambio de contexto:** El despachador guarda el estado actual del proceso que se estaba ejecutando (incluyendo el contador de programa y los

registros de la CPU) y carga el estado del nuevo proceso. Esto es esencial para que ambos procesos puedan reanudar su ejecución en el futuro sin perder información.

- **Conmutación a modo usuario:** El sistema operativo, que opera en modo privilegiado (modo kernel), cambia al modo de ejecución menos privilegiado (modo usuario) para permitir que el proceso de usuario comience su ejecución de manera segura.
- **Salto a la dirección de memoria correcta:** El despachador establece el contador de programa a la dirección de memoria adecuada en el código del nuevo proceso, permitiéndole reanudar su ejecución desde donde se detuvo.

El tiempo que le toma al despachador completar estas acciones, desde que detiene un proceso hasta que el siguiente comienza a ejecutarse, se conoce como **latencia de despacho**. Esta latencia es una sobrecarga inherente al sistema que se debe minimizar para optimizar el rendimiento.

1.2 Niveles de planificación de procesos

El objetivo de la planificación de procesos es asignar procesos a ser ejecutados por el procesador o procesadores a lo largo del tiempo, de forma que se cumplan los objetivos del sistema tales como el tiempo de respuesta, el rendimiento y la eficiencia del procesador. En muchos sistemas, esta actividad de planificación se divide en tres funciones independientes: planificación a largo-, medio-, y corto-plazo. Los nombres sugieren las escalas de tiempo relativas con que se ejecutan estas funciones.[2]

Estos tres niveles principales trabajan en conjunto para controlar el flujo de procesos en el sistema. Los veremos a continuación.

1.2.1 Planificación a largo plazo (Long-Term Scheduling)

Este es el nivel más alto de planificación. Su función principal es decidir **qué procesos, de los almacenados en el disco, se cargarán en la memoria principal** para convertirse en un proceso en espera de ejecución. El planificador a largo plazo tiene como objetivo controlar el grado de multiprogramación (el número de

[2] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

procesos en la memoria al mismo tiempo) para garantizar que el sistema no se sature. Si el grado de multiprogramación es muy alto, el rendimiento puede degradarse debido al excesivo cambio de contexto. Si es muy bajo, la CPU podría estar inactiva. Este planificador se ejecuta con poca frecuencia.

1.2.2 Planificación a medio plazo (Medium-Term Scheduling)

Este nivel se encarga de gestionar el intercambio de procesos entre la memoria principal y el disco. Su función principal es el **swapping** o **suspensión de procesos**. Si la memoria se llena demasiado, el planificador a medio plazo puede mover un proceso parcialmente ejecutado de la memoria principal al disco duro (swap-out) para liberar espacio. Más tarde, puede volver a cargarlo en la memoria (swap-in) para que continúe su ejecución. El objetivo es liberar espacio en la memoria principal para los procesos que están listos para ejecutarse y mejorar la eficiencia del sistema.

1.2.3 Planificación a corto plazo (Short-Term Scheduling) / Despachador (Dispatcher)

Este es el nivel más importante y se ejecuta con mayor frecuencia. Su tarea es decidir **qué proceso de la cola de 'listos' obtendrá la CPU** a continuación. Es el componente que toma las decisiones instantáneas de asignación de CPU. Este planificador debe ser muy rápido para minimizar la sobrecarga.

Repasemos brevemente la función del despachador (dispatcher)

El "despachador" es un componente fundamental en la planificación de la CPU. Es el módulo que asigna el control de la CPU a los procesos que han sido seleccionados por el **planificador a corto plazo**. Su trabajo implica:

- **Cambio de contexto:** Salvar el estado del proceso actual y cargar el del nuevo.
- **Cambio al modo de usuario:** Mover la ejecución del sistema operativo al modo de usuario.
- **Reinicio del programa:** Saltar a la dirección de memoria adecuada para que el programa pueda continuar su ejecución.

El despachador debe ser extremadamente rápido, ya que se invoca en cada cambio de proceso. El tiempo que tarda en detener un proceso y empezar otro se conoce como **latencia de despacho**.

Tabla comparativa de los niveles de planificación de procesos:

Nivel de Planificación	Función Principal	Objetivos y Comentarios	Frecuencia de Ejecución
Largo Plazo (Long-Term)	Decide qué procesos del disco se admiten en la memoria principal.	Controla el grado de multiprogramación para evitar la sobrecarga del sistema.	Baja (se ejecuta infrecuentemente).
Medio Plazo (Medium-Term)	Gestiona el movimiento de procesos entre la memoria principal y el disco (swapping).	Mejora la eficiencia al liberar espacio en la memoria principal para los procesos listos para ejecutarse.	Media (cuando la memoria se llena).
Corto Plazo (Short-Term)	Elige qué proceso de la cola de listos obtendrá la CPU.	Maximiza la utilización de la CPU y el rendimiento del sistema; implementa los algoritmos de planificación.	Alta (se ejecuta con mucha frecuencia).

1.2.4 Ejemplo de flujo de planificación

Un ejemplo práctico es la mejor manera de entender cómo los tres niveles de planificación trabajan juntos. Usaremos un escenario con una impresora para ilustrar cómo un trabajo se mueve por el sistema.

1. **Planificación a largo plazo:** Imagina que tienes 100 trabajos de impresión esperando en el disco duro. El planificador a largo plazo decide que sólo 20 de ellos se cargarán en la memoria principal para ser procesados. Su decisión se basa en la capacidad del sistema y el tipo de trabajos para evitar una sobrecarga, controlando el **grado de multiprogramación**.
2. **Planificación a medio plazo:** De los 20 trabajos en memoria, 10 están en espera de que la impresora termine. Si llegan 5 trabajos nuevos con una alta prioridad que necesitan un espacio en la memoria para comenzar, el planificador a medio plazo puede mover temporalmente algunos de los 10 trabajos en espera de la memoria al disco. Esto libera espacio para que los trabajos de alta prioridad se carguen y puedan ser atendidos más rápidamente. Una vez que se libera el espacio necesario, los trabajos que se movieron al disco pueden ser "devueltos" a la memoria para continuar.

3. **Planificación a corto plazo:** El planificador a corto plazo está constantemente activo, decidiendo qué proceso de los que están en la memoria obtendrá la CPU. Si un trabajo de impresión necesita hacer cálculos complejos antes de enviar el documento a la impresora, el planificador le asignará la CPU. Cuando el trabajo termine de calcular y esté listo para imprimir, entrará en un estado de espera de E/S. En ese momento, el planificador de corto plazo elegirá el siguiente trabajo de la cola de listos y le dará la CPU, evitando que esta se quede inactiva.

Este ciclo muestra cómo cada nivel de planificación tiene un rol específico para asegurar que el sistema sea eficiente y responda a las necesidades de los usuarios.

1.3 Algoritmos de Planificación de Procesos

Los **algoritmos de planificación de procesos** son los procedimientos paso a paso que aplican las reglas que utiliza el sistema operativo para determinar qué proceso debe ejecutarse a continuación en la CPU. Su objetivo principal es coordinar la ejecución de los procesos de manera organizada y eficiente, garantizando que ningún proceso se adueñe de la CPU.

Para ello, el **planificador de la CPU** (CPU Scheduler) selecciona de la cola de procesos listos al próximo a ser ejecutado. La estrategia que utiliza para esta selección es, precisamente, el algoritmo de planificación.

Los objetivos clave de un buen algoritmo de planificación de procesos son:

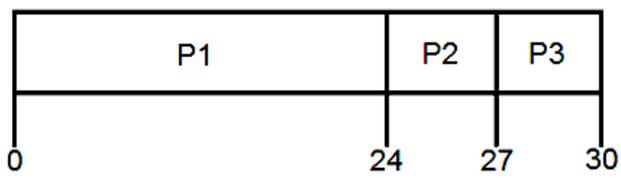
- **Maximizar la utilización de la CPU:** Mantener la CPU tan ocupada como sea posible para evitar tiempos de inactividad.
- **Maximizar la productividad (*throughput*):** Aumentar la cantidad de procesos que se completan en un período de tiempo determinado.
- **Minimizar el tiempo de espera:** Reducir el tiempo total que un proceso pasa en la cola de listos, esperando a ser ejecutado.
- **Minimizar el tiempo de retorno (*turnaround time*):** Disminuir el tiempo total que tarda un proceso en completarse, desde su llegada hasta su finalización.
- **Minimizar el tiempo de respuesta:** Acortar el tiempo que transcurre desde que un usuario realiza una solicitud hasta que recibe la primera respuesta, lo cual es crítico en sistemas interactivos.

A continuación veremos los algoritmos de planificación más comunes:

1.3.1 FCFS (First-Come, First-Served)

Este es el algoritmo más sencillo. El planificador ejecuta los procesos en el mismo orden en que llegan a la cola de listos (FIFO). Es justo en el sentido de que nadie "se salta la fila," pero puede ser ineficiente si un proceso muy largo llega primero y hace esperar a muchos procesos cortos. El esquema a continuación muestra el problema con un ejemplo de 3 procesos:

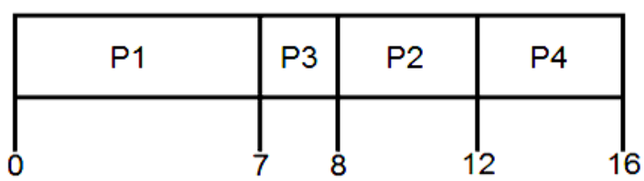
Procesos	llegada	Tiempo uso CPU
P1	0	24
P2	2	3
P3	4	3



1.3.2 SJF (Shortest Job First)

En este algoritmo , se prioriza a los procesos más cortos a la hora de la ejecución y los coloca en la cola. Selecciona el proceso con el próximo tiempo de ejecución más corto y lo ejecuta hasta que finaliza el proceso. Si hay dos procesos cuyas ráfagas de la CPU tiene la misma duración, se emplea el algoritmo FCFS.

Procesos	llegada	Tiempo uso CPU
P1	0	7
P2	2	4
P3	4	1
P4	5	4



Ejemplo de SJF (Shortest Job First)

En el ejemplo anterior, el proceso **P1** llega en el momento **0** y tiene un tiempo de

uso de CPU de **7**. Como es el único proceso disponible en ese momento, el planificador lo selecciona para su ejecución de inmediato. El planificador no puede "saber" que un proceso más corto llegará más tarde.

P1 se ejecuta de forma ininterrumpida hasta el final, en el momento **7**. Para ese momento, los procesos **P2** (llegó en 2), **P3** (llegó en 4) y **P4** (llegó en 5) ya han llegado y están en la cola de listos. En ese momento **7**, el planificador evalúa los procesos que están listos (**P2, P3, P4**) y selecciona el que tiene el tiempo de uso de CPU más corto. En este caso, **P3** (con un tiempo de 1) es el más corto. Después, le siguen **P2** y **P4** (con un tiempo de 4).

1.3.2 SRTF (Shortest Remaining Time First)

El algoritmo **SRTF** (Shortest Remaining Time First) es una versión **apropiativa** del SJF (Shortest Job First). Este algoritmo toma la decisión de qué proceso ejecutar a continuación cada vez que un proceso nuevo llega a la cola de listos. En ese momento, compara el tiempo restante de ejecución del proceso que está actualmente en la CPU con el tiempo total de CPU del nuevo proceso. Si el nuevo proceso tiene un tiempo de CPU más corto, el sistema operativo interrumpe (o apropia) al proceso en ejecución y comienza a ejecutar el nuevo.

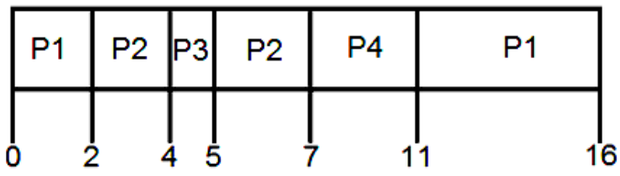
¿Qué lo diferencia del SJF no apropiativo?

La diferencia fundamental está en la **apropiación**. Mientras que el SJF no apropiativo solo toma decisiones cuando un proceso termina, el SRTF lo hace en dos momentos clave:

1. Cuando un proceso finaliza o cambia de estado.
2. Cada vez que un nuevo proceso llega a la cola de listos.

Este comportamiento le permite al SRTF ser más eficiente y reducir el tiempo de espera promedio de los procesos, ya que puede dar prioridad a un proceso más corto que llegue más tarde. Sin embargo, tiene un costo: la sobrecarga de tener que cambiar de contexto cada vez que un proceso más corto llega a la cola de listos.

Procesos	llegada	Tiempo uso CPU
P1	0	7
P2	2	4
P3	4	1
P4	5	4



Ejemplo de SRTF (Shortest Remaining Time First)

En el ejemplo del esquema anterior, la tabla muestra cuatro procesos (P1, P2, P3 y P4) con sus respectivos tiempos de llegada y tiempos de uso de CPU.

- **T = 0:** El **Proceso P1** llega y comienza a ejecutarse, ya que es el único proceso en la cola.
- **T = 2:** Llega el **Proceso P2**. P1 tiene 5 unidades de tiempo restantes (7 - 2 = 5) mientras que P2 tiene 4. Como P2 tiene un tiempo restante menor, el planificador lo selecciona. P1 se interrumpe y P2 comienza a ejecutarse.
- **T = 4:** Llega el **Proceso P3**. P2 tiene 2 unidades de tiempo restantes (4 - 2 = 2) y P3 tiene 1. P3 tiene el tiempo restante más corto, por lo que el planificador interrumpe a P2 y P3 comienza a ejecutarse.
- **T = 5:** P3 finaliza su ejecución, ya que solo necesitaba 1 unidad de tiempo. En este punto, llega el **Proceso P4** (con 4 unidades de tiempo). La cola de listos ahora tiene a P2 (con 2 unidades restantes), P4 (con 4 unidades) y P1 (con 5 unidades). De estos, el que tiene el menor tiempo restante es P2, por lo que este se reanuda.
- **T = 7:** P2 finaliza su ejecución (2 unidades de tiempo). Ahora, los procesos restantes son P4 (4 unidades restantes) y P1 (5 unidades restantes). P4 es el que tiene el menor tiempo restante y comienza a ejecutarse.
- **T = 11:** P4 finaliza su ejecución (4 unidades de tiempo). El único proceso restante es P1, que tenía 5 unidades de tiempo restantes. P1 se reanuda.
- **T = 16:** P1 finaliza su ejecución, ya que usa las 5 unidades de tiempo restantes, y todos los procesos han terminado.

1.3.3 RR (Round Robin)

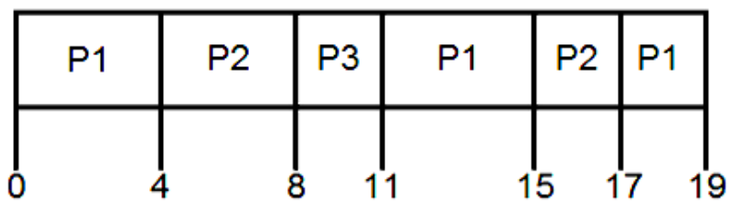
Este algoritmo de planificación es uno de los más complejos y difíciles de implementar, asigna a cada proceso un tiempo equitativo tratando a todos los procesos por igual y con la misma prioridad. Es un algoritmo circular, volviendo siempre al primer proceso una vez terminado con el último, para controlar este método a cada proceso se le asigna un intervalo de tiempo llamado quantum o cuanto.

Pueden suceder dos casos con este método:

- El proceso es menor que el quantum: Al terminar antes se planifica un nuevo proceso.
- El proceso es mayor que el quantum: Al terminar el quantum se expulsa el proceso dando paso al siguiente proceso en la lista. Al terminar la iteración se volverá para terminar el primer proceso expulsado.

Ejemplo con *quantum* = 4:

Procesos	llegada	Tiempo uso CPU
P1	0	10
P2	1	6
P3	2	3



Ejemplo de RR (Round Robin)

En el ejemplo del esquema anterior, la tabla muestra tres procesos: P1, P2 y P3. El quantum es 4. Analicemos cómo funciona:

- T = 0: El proceso P1 llega y comienza a ejecutarse. Se le asigna el primer quantum de 4.
- T = 4: El quantum de P1 termina. P1 aún necesita más tiempo de CPU (10 - 4 = 6). El planificador lo interrumpe y lo coloca al final de la cola de listos. En

este punto, P2 y P3 ya han llegado, por lo que la cola de listos es P2, P3, P1. El planificador elige a P2 para ejecutar.

- T = 8: El quantum de P2 termina. P2 aún necesita más tiempo (6 - 4 = 2). El planificador lo interrumpe y lo coloca al final de la cola de listos. La cola ahora es P3, P1, P2. El planificador elige a P3 para ejecutar.
- T = 11: P3 solo necesitaba 3 unidades de tiempo (su tiempo de uso de CPU era 3). Como su tiempo de uso es menor que el quantum, P3 termina su ejecución. La cola ahora es P1, P2. El planificador elige a P1 para su segundo turno.
- T = 15: El quantum de P1 termina. P1 aún necesita más tiempo (6 - 4 = 2). El planificador lo interrumpe y lo coloca al final de la cola. La cola ahora es P2, P1. El planificador elige a P2 para su segundo turno.
- T = 17: P2 solo necesita 2 unidades de tiempo para terminar (su tiempo de uso restante era 2). P2 finaliza. La cola ahora es solo P1. El planificador elige a P1 para su último turno.
- T = 19: P1 usa sus últimas 2 unidades de tiempo (su tiempo restante era 2). P1 finaliza su ejecución. El diagrama de Gantt termina en T = 19.

1.3.4 Planificación por prioridades

El algoritmo de **planificación por prioridad** es un método en los sistemas operativos que asigna un valor de prioridad a cada proceso. El planificador de la CPU se encarga de ejecutar siempre el proceso que tiene la **prioridad más alta**.

Funcionamiento

- **Asignación de Prioridad:** A cada proceso se le asigna un número entero que representa su prioridad. Este valor puede ser asignado de forma **estática** (permanece fijo) o **dinámica** (cambia con el tiempo). En muchos sistemas, un número más pequeño corresponde a una prioridad más alta.
- **Selección del Proceso:** El planificador evalúa la cola de procesos listos y elige el que tiene el valor de prioridad más favorable (por ejemplo, el número más bajo).
- **Tipos de Prioridad:**
    - **Prioridad Estática:** La prioridad se define al crear el proceso y no cambia.

- **Prioridad Dinámica:** La prioridad puede ajustarse durante la ejecución. Por ejemplo, la prioridad de un proceso que ha esperado mucho tiempo puede aumentar para evitar que se quede sin ejecutar indefinidamente.

Variantes del Algoritmo

- **No Expropiativo (Non-Preemptive):** Una vez que un proceso comienza a ejecutarse, mantiene la CPU hasta que finaliza o se bloquea (por ejemplo, al solicitar una operación de E/S). Si llega un proceso con mayor prioridad, este debe esperar a que el proceso actual termine.
- **Expropiativo (Preemptive):** Este es el tipo más común. Si un proceso nuevo con mayor prioridad llega mientras otro se está ejecutando, el proceso actual se interrumpe y se mueve a la cola de listos para dar paso al proceso de mayor prioridad.

Ventajas y Desventajas

Ventajas

- **Respuesta rápida para procesos importantes:** Garantiza que las tareas críticas o sensibles al tiempo se ejecuten rápidamente.
- **Flexibilidad:** Permite al administrador del sistema dar un trato preferencial a ciertas tareas.

Desventajas

- **Inanición (Starvation):** Es el problema más grave. Si hay un flujo constante de procesos de alta prioridad, los procesos de baja prioridad pueden quedarse esperando indefinidamente y nunca llegar a ejecutarse.
- **Solución:** Para mitigar la inanición, se usa una técnica llamada **"envejecimiento"** (aging), que aumenta gradualmente la prioridad de los procesos que han estado esperando por mucho tiempo.

1.4 Planificación de procesos en sistemas multiprocesador

La **planificación de procesos** es una función esencial de los sistemas operativos, encargada de decidir qué proceso se ejecuta, cuándo y en qué procesador. En sistemas multiprocesador —aquellos con dos o más unidades de procesamiento— esta tarea se vuelve más compleja y crítica, ya que debe considerar la distribución eficiente de la carga de trabajo entre múltiples CPUs, la sincronización entre procesos, y la minimización de tiempos de espera y latencia.

1.4.1 Objetivos de la planificación en multiprocesadores

- **Asignación de procesos a CPUs**: Decidir qué proceso se ejecuta en qué procesador.
- **Balanceo de carga**: Evitar que algunos procesadores estén sobrecargados mientras otros están inactivos.
- **Sincronización y coherencia de caché**: Mantener la consistencia de datos entre procesadores.
- **Afinidad de procesador**: Preferencia por ejecutar un proceso en el mismo procesador para aprovechar la caché.
- **Interrupciones distribuidas**: Las interrupciones pueden ser manejadas por cualquier procesador, lo que requiere coordinación.

1.4.2 Estrategias de planificación

En un entorno multiprocesador, existen dos enfoques principales para gestionar los procesos:

- **Cola de procesos única**: Se mantiene una sola lista de procesos para todos los procesadores. Los procesos se asignan a cualquier procesador que esté disponible. Este método trata a los procesadores como unidades de ejecución idénticas e intercambiables.
- **Colas de procesos separadas**: Cada procesador tiene su propia lista de procesos. Esto permite una mayor especialización o control sobre qué procesos se ejecutan en cada núcleo, pero puede complicar el equilibrio de carga.

Ahora veremos dos conceptos clave para la planificación en sistemas multiprocesador: afinidad al procesador y equilibrio de carga. Ambos influyen directamente en el rendimiento, la eficiencia y la experiencia del usuario en sistemas modernos.

1.4.3 Afinidad al procesador

La **afinidad al procesador** se refiere a la preferencia o vínculo entre un proceso (o hilo) y un procesador específico. Esta relación busca aprovechar la **localidad de caché**, es decir, los datos que permanecen en la memoria caché del procesador tras la ejecución de un proceso.

Tipos de afinidad

- **Afinidad suave (soft affinity)**: El sistema operativo intenta mantener el proceso en el mismo procesador, pero puede reasignarlo si es necesario.
- **Afinidad rígida (hard affinity)**: El proceso está estrictamente vinculado a un procesador específico.

Ventajas

- **Mejora del rendimiento**: Al mantener los datos en la caché del mismo procesador, se reducen los accesos a memoria principal.
- **Reducción de fallos de caché**: Menos cambios de contexto entre procesadores disminuyen las pérdidas de datos temporales.

Desventajas

- **Menor flexibilidad**: Si el procesador está ocupado, el proceso puede quedarse esperando innecesariamente.
- **Complicaciones en arquitecturas NUMA**: En sistemas con acceso no uniforme a memoria, la afinidad debe considerar la topología de memoria compartida.

1.4.4 Equilibrio de carga

El **equilibrio de carga** es una técnica esencial que busca distribuir los procesos de manera equitativa entre todos los procesadores disponibles. Esto evita que

algunos procesadores se sobrecarguen mientras otros permanecen infrautilizados. Existen diferentes **estrategias comunes** para lograr este objetivo. Por un lado, el **balanceo estático** asigna los procesos al inicio del sistema y esta asignación no cambia, lo que lo hace simple, pero poco adaptable a las variaciones de la carga. Por otro lado, el **balanceo dinámico** es una estrategia más avanzada donde el sistema operativo monitoriza constantemente la carga de los procesadores y puede **migrar procesos** entre CPUs para optimizar su uso.

Sin embargo, hay **consideraciones clave** que se deben tener en cuenta. Aunque la migración de procesos puede mejorar la distribución de la carga, también puede generar penalizaciones por la pérdida de caché. Además, la **topología del sistema** es un factor importante, especialmente en sistemas NUMA (Non-Uniform Memory Access). En estos casos, el balanceo de carga debe considerar la cercanía de la memoria al procesador para evitar latencias elevadas, garantizando así un rendimiento óptimo.

Pensemos en el siguiente ejemplo:

Imagina dos procesos intensivos (A y B) asignados al mismo procesador, mientras otro procesador está libre. El sistema puede mover el proceso B al procesador desocupado, logrando un mejor uso de recursos y reduciendo el tiempo de espera.



Actividad S6-A1 (Obligatoria)

Realice la actividad identificada como *S6-A1* (Semana 6, Actividad 1) en la sección de actividades del campus.